

CS4782 - Low-Rank Adaptation of Large Language Models (LoRA)

Kyle Chu (kgc42), Gabriel Huang (gzh2)

1. Introduction

Full fine-tuning of pretrained language models requires updating and storing a separate copy of every parameter for each downstream task, which is prohibitively expensive as model sizes scale. *LoRA: Low-Rank Adaptation of Large Language Models* (Hu, Shen, Wallis, Allen-Zhu, Li, Wang, Wang, Chen, 2021) introduces a parameter-efficient alternative that freezes the pretrained weights W_0 and learn a

low-rank update $\Delta W = BA$, where $A \in \mathbb{R}^{r \times d}$ and $B \in \mathbb{R}^{d \times r}$ with rank $r \ll d$. A is initialized from a random distribution, B is initialized to zero, and the adapted forward pass is $h = W_0 x + (\alpha/r) \cdot BAx$. The paper's central claim is that this low-rank update matches or exceeds full fine-tuning on GLUE with only a small fraction of the trainable parameters, while also reducing optimizer memory and training times. Beyond accuracy, LoRA introduces no additional inference latency, enables cheap task-switching by swapping only the A and B matrices, and is orthogonal to existing methods such as prefix tuning.

2. Chosen Result

We reproduce the validation accuracy of LoRA versus full fine-tuning on SST-2, QNLI, and RTE from the RoBERTa-base rows of Table 2 of the paper. This result is the headline empirical evidence for the paper's main claim on an encoder-only model, and it is reproducible at our compute budget. It lets us verify both the quality claim (matching accuracy) and the efficiency claim (parameter count, training times, peak GPU memory) from a single set of runs. We fix rank $r = 8$ and scaling $\alpha = 8$ for LoRA, matching the hyperparameters the paper reports for these tasks.

3. Methodology

For the model architecture, we load roberta-base (124.6M params) via HuggingFace Transformers and implement a custom LoRALinear module from scratch. It wraps an nn.Linear with a frozen base weight and a trainable low-rank pair (A, B) . The forward pass is written in PyTorch's row-vector convention as $x \cdot W_0^T + b_0 + (\alpha/r) \cdot (x \cdot A^T) B^T$, with B zero-initialized so that the adapter contributes nothing at step 0 and the model reproduces the pretrained baseline exactly. Following the paper, we inject LoRA into the query and value projections of every self-attention block only. All other weights (embeddings, MLPs, LayerNorms, K/O projections) are frozen with `requires_grad=False`. The classifier head is trainable.

We use the GLUE SST-2, QNLI, and RTE splits from HuggingFace Datasets, tokenized with the RoBERTa BPE tokenizer at max sequence length 512, exactly like the paper. We report validation accuracy (best across epochs), trainable parameter count, peak GPU memory, and epoch training time.

We used the same hyperparameters and optimizers from the paper for our training. We used AdamW with decoupled weight decay (0.01 on weights; 0.0 on biases and LayerNorm), linear warmup over 6% of steps, gradient clipping at 1.0. LoRA learning rate is $5e-4$ for SST-2 and RTE and $4e-4$ for QNLI. Full-FT learning rate is $2e-5$. Batch size is 16 for SST-2 and 32 for QNLI/RTE. To validate correctness we include a `verify_lora_model` function that asserts only A , B , and the classifier carry gradients, no base weight has a non-None `.grad` after a backward pass, and the LoRA-wrapped model's logits equal the plain pretrained model's logits at init (since $B = 0$).

We deviated from the paper in a couple of ways. We reduce epochs for SST-2 from the paper's 60 to 10 and QNLI from 25 to 10 to stay within the compute budget. RTE is trained for 80 epochs as in the paper. For RTE we initialize A and B from scratch rather than warm-starting from an MNLI-adapted LoRA checkpoint. We use Kaiming-uniform initialization for A instead of the paper's $N(0, 1)$. In our setup the unit-variance initialization caused exploding activations at step 0 that prevented B from escaping its zero initialization. We also took the best validation accuracy from only one training run, while the paper took the median validation accuracy from 5 training runs for SST-2, QNLI, and RTE.

4. Results & Analysis

Task	Full FT (ours)	Full FT (paper)	Trainable params for Full FT	LoRA (ours)	LoRA (paper)	Trainable params for LoRA
SST-2	93.69	94.8	125M	94.38	95.1	0.294M
QNLI	92.90	92.8	125M	92.81	93.3	0.294M
RTE	79.42	78.7	125M	80.14	86.6*	0.294M

Table 1. Validation accuracy (%) on GLUE tasks. LoRA uses $r=8$ applied to W_q and W_v . *Paper warm-starts RTE from an MNLI-adapted checkpoint; our RTE LoRA is initialized from scratch

Task	Peak memory LoRA (MB)	Peak memory Full FT (MB)	Average time/epoch (seconds) LoRA	Average time/epoch (seconds) Full FT
SST-2	4,206	6,603	907.8	1,275.7
QNLI	7,908	11,751	583.7	727.5
RTE	7,908	11,698	76.4	96.3

Table 2. Peak GPU memory and training time per epoch for LoRA vs. full fine-tuning on RoBERTa-base. These measurements are an original contribution of this re-implementation. Used L4 GPU for RTE, A100 High-Ram for SST-2 and QNLI.

LoRA trains 294,912 parameters versus 124,647,170 for full fine-tuning (about 0.24% as many) yet reaches comparable accuracy (Table 1). Peak GPU memory drops by $\sim 33\text{--}36\%$, driven mostly by AdamW’s per-parameter m/v optimizer states being allocated only for trainable tensors. LoRA also ran $\sim 20\%$ faster per epoch (Table 2).

Our LoRA scores come within ~ 0.5 pts of the paper on QNLI (92.81 vs 93.3) and 0.7 pts on SST-2 (94.38 vs 95.1) despite 6x fewer epochs on SST-2 and 2.5x fewer on QNLI. Our full-FT QNLI (92.90) actually slightly exceeds the paper’s reported 92.8. The clear outlier is RTE-LoRA (80.14 vs 86.6). The paper notes that RTE are initialized from an MNLI LoRA checkpoint rather than from scratch, a setting we could not afford to reproduce. Our full-FT RTE and QNLI is above the paper’s RTE and QNLI respectively, which we can attribute the difference due to the fact that the paper draws their number from the median validation accuracy from 5 training runs for SST-2, QNLI, and RTE.

On QNLI, full fine-tuning overfits fast (train loss $0.37 \rightarrow 0.02$ in 10 epochs while val accuracy plateaus at ~ 92.9), whereas LoRA’s train loss stays above 0.17 and its validation curve keeps creeping up which is consistent with the paper’s framing of low-rank adaptation as an implicit regularizer. On RTE, LoRA takes ~ 25 epochs to escape the majority-class accuracy plateau and its validation curve is noisy (± 2 pts epoch-to-epoch), highlighting how small-data GLUE tasks amplify seed/initialization variance and why the paper’s MNLI warm-start matters.

5. Reflection

One takeaway is the importance of initialization for training. Swapping A’s init from $N(0,1)$ to Kaiming-uniform was the single change that unblocked training. Any re-implementation of LoRA should explicitly verify step-0 equivalence to the base model and monitor the magnitude of ΔW_x .

Another thing is that the learning rate must be re-tuned. LoRA’s optimal LR ($5e-4$) is $\sim 20\times$ larger than full-FT’s ($2e-5$). Updating $\sim 0.7\%$ of the parameters means each step barely moves the loss unless the LR compensates.

The biggest lesson was a reality check on compute budget. An 80-epoch RTE LoRA run took ~1.7 hours on an L4; the SST-2 full-FT run took ~3.5 hours on an A100. Reproducing Table 2 at paper scale (60 epochs SST-2, multiple seeds) would have consumed all of our remaining Colab credits which is why we reduced the number of epochs and rank sweep.

For future work, we could add the MNLI→RTE warm-start to close the RTE gap. Sweep $r \in \{1, 2, 4, 8, 16, 32, 64\}$ to see which ranks are most effective. Apply LoRA to a larger pre-trained model to test the scaling. Quantify adapter-swap latency for multi-task serving, a key benefit to LoRA in inference time.

6. References

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, L., Wang, W., & Chen, W. (2021). *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv:2106.09685.

Wang, A., Singh, A., Michael, J., Hill, F., Levy, O., & Bowman, S. R. (2019). *GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding*. ICLR.

PyTorch: <https://docs.pytorch.org/docs/stable/index.html>

GLUE Benchmark (HuggingFace Datasets): <https://huggingface.co/datasets/nyu-mll/glue>

HuggingFace Transformers: <https://huggingface.co/FacebookAI/roberta-base>